Data Structures and Algorithms

BEGINNER LEVEL - Complete Learning Guide

Target Audience: Complete beginners with basic programming knowledge
Time to Complete: 4-6 weeks
Prerequisites: Basic understanding of variables, loops, and functions
Version: 1.0

Table of Contents

Module 1: Foundations (Weeks 1-2)

1. Introduction to DSA and Problem Solving
2. Essential Mathematics for Beginners
3. Understanding Time and Space Complexity
4. Basic Problem-Solving Techniques

Module 2: Basic Data Structures (Weeks 3-4)

5. Arrays - Your First Data Structure
6. Strings and String Manipulation
7. Introduction to Linked Lists
8. Stacks - Last In, First Out

Module 3: Basic Algorithms (Weeks 5-6)

9. Queues - First In, First Out
10. Simple Searching Algorithms
11. Basic Sorting Algorithms
12. Introduction to Recursion

Appendices

- Beginner’s Big O Cheat Sheet
- 50 Easy Practice Problems
- Common Mistakes to Avoid
- Next Steps to Intermediate Level

Module 1: Foundations

Chapter 1: Introduction to DSA and Problem Solving

1.1 Welcome to the World of DSA!

What You'll Learn: By the end of this guide, you'll be able to: - Write efficient code that scales - Solve coding interview problems - Understand how data is organized in memory - Think like a programmer

Your Learning Journey:

```
Week 1-2: Learn the basics
   ↓
Week 3-4: Practice with data structures
   ↓
Week 5-6: Master simple algorithms
   ↓
Ready for Intermediate Level!
```

1.2 What is Computer Science?

Imagine you have 1000 books and need to find one specific book. You have three options:

Option 1: Random Search - Pick books randomly - Could take forever! - Time: Unpredictable

Option 2: Linear Search - Check every book one by one - Takes about 500 tries on average - Time: Proportional to number of books

Option 3: Organized System - Arrange books alphabetically - Open to middle, go left or right - Takes only ~10 tries for 1000 books! - Time: Much faster!

This is what Computer Science is about: Finding the best way to solve problems!

1.3 What are Data Structures?

A **data structure** is like a container that holds your data in a specific way.

Real-World Analogy:

 Stack of Plates - Add plates on top - Remove from top only - Last plate in = First plate out - This is a STACK!

 Library Card Catalog - Each card points to book location - Quick lookup by name - This is a HASH TABLE!

 Train of Carriages - Each carriage connects to next - Can add/remove carriages easily - This is a LINKED LIST!

Why Different Structures? - Some are fast for searching - Some are fast for adding/removing -

Some use less memory - Choose the right tool for the job!

1.4 What are Algorithms?

An **algorithm** is a recipe for solving a problem.

Example: Making a Sandwich

```
Algorithm: MakePeanutButterSandwich

1. Get two slices of bread
2. Get peanut butter jar
3. Get knife
4. Open peanut butter jar
5. Spread peanut butter on one slice
6. Put slices together
7. Done!
```

Coding Example: Find the Largest Number

```
Input: [3, 7, 2, 9, 1, 5]

Algorithm:
1. Assume first number is largest (3)
2. Compare with second number (7 > 3, so now 7 is largest)
3. Compare with third number (2 < 7, so 7 still largest)
4. Compare with fourth number (9 > 7, so now 9 is largest)
5. Continue...
6. Return 9

Output: 9
```

In Code (Python):

```python
def find_largest(numbers):
    # Start with first number
    largest = numbers[0]

    # Compare with each number
    for number in numbers:
        if number > largest:
            largest = number

    return largest

# Test it
numbers = [3, 7, 2, 9, 1, 5]
result = find_largest(numbers)
print(result)  # Output: 9
```

In Code (JavaScript):

```
function findLargest(numbers) {
    // Start with first number
    let largest = numbers[0];

    // Compare with each number
    for (let number of numbers) {
        if (number > largest) {
            largest = number;
        }
    }

    return largest;
}

// Test it
let numbers = [3, 7, 2, 9, 1, 5];
let result = findLargest(numbers);
console.log(result);  // Output: 9
```

1.5 Why Study DSA?

Reason 1: Write Better Code

```
# Bad: Check every pair (slow for large lists)
def has_duplicates_slow(arr):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False

# Good: Use a set (much faster!)
def has_duplicates_fast(arr):
    seen = set()
    for num in arr:
        if num in seen:
            return True
        seen.add(num)
    return False
```

Reason 2: Solve Real Problems - Netflix: Recommend movies you'll like - Google Maps: Find fastest route - Instagram: Show posts in order of relevance - All use DSA!

Reason 3: Get a Job - Most tech interviews test DSA - Companies like Google, Amazon, Microsoft require it - Even startups ask DSA questions

Reason 4: Think Logically - Break big problems into small steps - See patterns in problems - Develop systematic thinking

1.6 Programming Basics Review

Before we dive deeper, let's review essential programming concepts.

Variables: Containers for Data

```python
# Python
name = "Alice"           # String (text)
age = 25                 # Integer (whole number)
height = 5.6             # Float (decimal number)
is_student = True        # Boolean (true/false)
```

```javascript
// JavaScript
let name = "Alice";      // String
let age = 25;            // Number
let height = 5.6;        // Number
let isStudent = true;    // Boolean
```

Conditional Statements: Making Decisions

```python
# Python
age = 20

if age >= 18:
    print("You can vote!")
elif age >= 16:
    print("You can drive!")
else:
    print("You're young!")
```

```javascript
// JavaScript
let age = 20;

if (age >= 18) {
    console.log("You can vote!");
} else if (age >= 16) {
    console.log("You can drive!");
} else {
    console.log("You're young!");
}
```

Loops: Repeating Actions

```python
# Python - For loop
for i in range(5):
    print(i)  # Prints: 0, 1, 2, 3, 4

# Python - While loop
count = 0
while count < 5:
    print(count)
```

```
    count += 1
```

```
// JavaScript - For loop
for (let i = 0; i < 5; i++) {
    console.log(i);  // Prints: 0, 1, 2, 3, 4
}

// JavaScript - While loop
let count = 0;
while (count < 5) {
    console.log(count);
    count++;
}
```

Functions: Reusable Code Blocks

```
# Python
def greet(name):
    return f"Hello, {name}!"

def add_numbers(a, b):
    return a + b

# Using functions
message = greet("Alice")      # "Hello, Alice!"
sum_result = add_numbers(5, 3)  # 8
```

```
// JavaScript
function greet(name) {
    return `Hello, ${name}!`;
}

function addNumbers(a, b) {
    return a + b;
}

// Using functions
let message = greet("Alice");      // "Hello, Alice!"
let sumResult = addNumbers(5, 3);  // 8
```

Lists/Arrays: Collections of Items

```
# Python
fruits = ["apple", "banana", "orange"]

# Access items
first_fruit = fruits[0]     # "apple"
last_fruit = fruits[-1]     # "orange"
```

```python
# Add items
fruits.append("grape")      # ["apple", "banana", "orange", "grape"]

# Loop through items
for fruit in fruits:
    print(fruit)
```

```javascript
// JavaScript
let fruits = ["apple", "banana", "orange"];

// Access items
let firstFruit = fruits[0];           // "apple"
let lastFruit = fruits[fruits.length - 1];  // "orange"

// Add items
fruits.push("grape");  // ["apple", "banana", "orange", "grape"]

// Loop through items
for (let fruit of fruits) {
    console.log(fruit);
}
```

1.7 Your First Algorithm: Sum of Numbers

Let’s write our first complete algorithm!

Problem: Given a list of numbers, find their sum.

Step-by-Step Thinking: 1. Start with sum = 0 2. Look at first number, add to sum 3. Look at second number, add to sum 4. Continue for all numbers 5. Return the sum

Implementation:

```python
# Python
def sum_of_numbers(numbers):
    """Calculate sum of all numbers in a list"""
    total = 0  # Step 1: Start with 0

    # Steps 2-4: Add each number
    for number in numbers:
        total = total + number

    # Step 5: Return the result
    return total

# Test it
my_numbers = [1, 2, 3, 4, 5]
result = sum_of_numbers(my_numbers)
print(f"Sum: {result}")  # Output: Sum: 15
```

```javascript
// JavaScript
function sumOfNumbers(numbers) {
    // Calculate sum of all numbers in an array
    let total = 0;  // Step 1: Start with 0

    // Steps 2-4: Add each number
    for (let number of numbers) {
        total = total + number;
    }

    // Step 5: Return the result
    return total;
}

// Test it
let myNumbers = [1, 2, 3, 4, 5];
let result = sumOfNumbers(myNumbers);
console.log(`Sum: ${result}`);  // Output: Sum: 15
```

1.8 Problem-Solving Mindset

The 5-Step Process:

```
1. UNDERSTAND
   - What is the input?
   - What is the output?
   - What are the rules?

2. EXAMPLES
   - Try small examples
   - Think of edge cases

3. BREAK IT DOWN
   - Write steps in plain English
   - Draw diagrams if needed

4. CODE IT
   - Translate steps to code
   - Test as you go

5. TEST & FIX
   - Try different inputs
   - Fix bugs
```

Example Problem: Count Even Numbers

Step 1: Understand - Input: List of numbers - Output: Count of even numbers - Rule: Even means divisible by 2

Step 2: Examples

```
[1, 2, 3, 4, 5, 6] → 3 (numbers 2, 4, 6 are even)
[1, 3, 5] → 0 (no even numbers)
[2, 4, 6] → 3 (all even)
```

Step 3: Break It Down

```
1. Start with count = 0
2. For each number:
   a. Check if number % 2 == 0
   b. If yes, add 1 to count
3. Return count
```

Step 4: Code It

```python
# Python
def count_even_numbers(numbers):
    count = 0
    for number in numbers:
        if number % 2 == 0:  # Check if even
            count += 1
    return count
```

Step 5: Test

```python
# Test cases
print(count_even_numbers([1, 2, 3, 4, 5, 6]))  # Should be 3
print(count_even_numbers([1, 3, 5]))           # Should be 0
print(count_even_numbers([2, 4, 6]))           # Should be 3
```

Chapter 1 Summary

What You Learned: - DSA helps you write efficient code - Data structures organize data - Algorithms are step-by-step solutions - Problem-solving follows a 5-step process - Practice makes perfect!

Key Takeaways: 1. Start with understanding the problem 2. Try examples before coding 3. Break problems into small steps 4. Test your solution

Chapter 1 Exercises

Exercise 1: Average Calculator Write a function that calculates the average of numbers in a list.

```
Input: [10, 20, 30, 40]
Output: 25
```

Exercise 2: Find Minimum Write a function that finds the smallest number in a list.

```
Input: [5, 2, 8, 1, 9]
Output: 1
```

Exercise 3: Count Odd Numbers Write a function that counts odd numbers in a list.

```
Input: [1, 2, 3, 4, 5]
Output: 3
```

Exercise 4: Product of Numbers Write a function that multiplies all numbers in a list.

```
Input: [2, 3, 4]
Output: 24
```

Exercise 5: Number Exists? Write a function that checks if a number exists in a list.

```
Input: [1, 2, 3, 4, 5], target = 3
Output: True
```

Solutions are in Appendix D!

Chapter 2: Essential Mathematics for Beginners

2.1 Why Math Matters (Don't Worry!)

You don't need to be a math genius! We'll only use: - Basic arithmetic (add, subtract, multiply, divide) - Simple patterns - Counting

What We'll Learn: - How to count operations - Understanding growth patterns - Basic math needed for algorithms

2.2 Basic Arithmetic Operations

Addition and Subtraction

```python
# Python
a = 5 + 3   # 8
b = 10 - 4  # 6
c = a + b   # 14
```

Multiplication and Division

```python
# Python
x = 5 * 3   # 15
y = 20 / 4  # 5.0 (decimal division)
z = 20 // 4 # 5 (integer division)
```

Modulo (Remainder)

```python
# Python
remainder = 17 % 5  # 2 (17 ÷ 5 = 3 remainder 2)
```

```
# Useful for checking even/odd
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Why Modulo is Useful:

```
# Check if divisible by 3
if number % 3 == 0:
    print("Divisible by 3")

# Circular array (wrap around)
index = (current_index + 1) % array_length

# Get last digit
last_digit = number % 10
```

2.3 Powers and Square Roots (Simple Version)

Powers (Exponents)

Power means "multiply by itself multiple times"

```
2^1 = 2
2^2 = 2 × 2 = 4
2^3 = 2 × 2 × 2 = 8
2^4 = 2 × 2 × 2 × 2 = 16
```

In Code:

```
# Python
result = 2 ** 3  # 8 (2 to the power of 3)
result = pow(2, 3)  # Also 8
```

```
// JavaScript
let result = Math.pow(2, 3);  // 8
let result2 = 2 ** 3;         // 8
```

Why Powers Matter: - Computer memory: 2^10 = 1024 bytes (1 KB) - Fast algorithms grow like 2^n - Understanding algorithm speed

Square Roots

Square root asks: "What number × itself = this number?"

```
√4 = 2   (because 2 × 2 = 4)
√9 = 3   (because 3 × 3 = 9)
```

```
√16 = 4  (because 4 × 4 = 16)
```

In Code:

```python
# Python
import math
result = math.sqrt(16)  # 4.0
```

```javascript
// JavaScript
let result = Math.sqrt(16);  // 4
```

2.4 Understanding Logarithms (The Easy Way!)

What is a Logarithm?

Logarithm asks: "How many times do I multiply to reach this number?"

```
log₂(8) = 3    because 2 × 2 × 2 = 8 (multiply 3 times)
log₂(16) = 4   because 2 × 2 × 2 × 2 = 16 (multiply 4 times)
```

Visual Understanding:

```
Start: 1
×2 → 2
×2 → 4
×2 → 8
×2 → 16
×2 → 32

How many multiplications from 1 to 32?
5 multiplications, so log₂(32) = 5
```

Why Logarithms Matter in DSA:

Logarithms describe EFFICIENT algorithms!

```python
# Linear search: Check every item
def linear_search(arr, target):
    for item in arr:  # n operations
        if item == target:
            return True
    return False
# Time: Proportional to array size (n)

# Binary search: Cut in half each time
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
            return True
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
# Time: Proportional to log(n) - MUCH FASTER!
```

Real-World Example:

Finding a word in a dictionary: - **Linear way**: Check every word (20,000+ words!) - **Smart way**: Open to middle, go left or right (only ~15 checks!)

That's the power of logarithmic algorithms!

2.5 Counting Operations

When we analyze algorithms, we count operations.

Example 1: Simple Loop

```
def print_numbers(n):
    for i in range(n):  # Runs n times
        print(i)        # 1 operation each time

# Total operations: n
```

Example 2: Nested Loop

```
def print_pairs(n):
    for i in range(n):      # Runs n times
        for j in range(n):  # Runs n times for each i
            print(i, j)     # 1 operation each time

# Total operations: n × n = n²
```

Example 3: Two Separate Loops

```
def print_then_print_again(n):
    for i in range(n):    # n operations
        print(i)

    for i in range(n):    # n operations
        print(i)

# Total operations: n + n = 2n
```

Example 4: Halving

```
def halve_until_one(n):
```

```
    while n > 1:
        print(n)
        n = n // 2  # Cut in half

# Example: n=16 → 16, 8, 4, 2, 1
# Operations: log₂(n)
```

2.6 Patterns and Sequences

Pattern 1: Adding Numbers

```
1 + 2 + 3 + 4 + 5 = ?

Pattern: 1+5=6, 2+4=6, 3 is alone
Total: 6 + 6 + 3 = 15

Formula: n × (n+1) / 2
For n=5: 5 × 6 / 2 = 15
```

Pattern 2: Doubling

```
1, 2, 4, 8, 16, 32...
Each number is 2× the previous

Powers of 2: 2^0, 2^1, 2^2, 2^3...
```

Pattern 3: Fibonacci

```
Each number is sum of previous two:
0, 1, 1, 2, 3, 5, 8, 13, 21...

0 + 1 = 1
1 + 1 = 2
1 + 2 = 3
2 + 3 = 5
...
```

2.7 Simple Logic

AND (Both must be true)

```
age = 25
has_license = True

if age >= 18 and has_license:
    print("Can drive!")  # Both conditions true
```

OR (At least one must be true)

```python
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("Don't go to work!")  # At least one is true
```

NOT (Opposite)

```python
is_raining = False

if not is_raining:
    print("Go outside!")  # not False = True
```

2.8 Basic Counting Techniques

Counting Possibilities

Example: Picking Outfits

```
3 shirts (Red, Blue, Green)
2 pants (Jeans, Khakis)

Total outfits = 3 × 2 = 6

Red + Jeans
Red + Khakis
Blue + Jeans
Blue + Khakis
Green + Jeans
Green + Khakis
```

Example: Password Combinations

```
4-digit PIN with numbers 0-9

First digit: 10 choices
Second digit: 10 choices
Third digit: 10 choices
Fourth digit: 10 choices

Total: 10 × 10 × 10 × 10 = 10,000 combinations
```

Chapter 2 Summary

What You Learned: - Basic arithmetic for DSA - Powers and square roots - Logarithms (cutting in half) - Counting operations in code - Simple patterns - Basic logic (AND, OR, NOT)

Key Math Concepts: 1. Operations: +, -, ×, ÷, % (modulo) 2. Powers: 2^3 = 8 3. Logs: $\log_2(8) = 3$ 4. Count operations to analyze speed

Chapter 2 Exercises

Exercise 1: What is 2^5? Calculate by hand, then verify in code.

Exercise 2: How many times do you halve 64 to reach 1?

Exercise 3: Count operations:

```
def mystery(n):
    for i in range(n):
        for j in range(10):
            print(i, j)
```

Exercise 4: If you have 5 t-shirts and 3 pairs of shoes, how many outfit combinations?

Exercise 5: What is 25 % 7? (Use calculator or code)

Chapter 3: Understanding Time and Space Complexity

3.1 Why Do We Care About Speed?

Story Time:

```
You're building an app with 1 million users.

Algorithm A: Takes 1 second for 100 users
Algorithm B: Takes 1 second for 100 users

Both seem equal, right?

But with 1 million users:
Algorithm A: Takes 10,000 seconds (2.7 hours!)
Algorithm B: Takes 10 seconds

HUGE difference!
```

We need to predict how algorithms perform as data grows!

3.2 What is Time Complexity?

Time complexity = How runtime grows as input grows

We don't measure in seconds because: - Different computers are faster/slower - Same code runs at different speeds - We care about the PATTERN, not exact time

Example:

```
# Algorithm 1
def example1(arr):
    print(arr[0])  # Always 1 operation
```

```
# Algorithm 2
def example2(arr):
    for item in arr:
        print(item)  # n operations

# Algorithm 3
def example3(arr):
    for i in arr:
        for j in arr:
            print(i, j)  # n × n operations
```

As array size doubles: - Algorithm 1: Same time (1 operation) - Algorithm 2: Double time (2n operations) - Algorithm 3: 4× time ($4n^2$ operations)

3.3 Big O Notation (Simple Explanation)

Big O tells us how an algorithm grows in the WORST CASE.

Think of it as a speed limit sign: - O(1): Always same speed (constant) - O(n): Speed depends on distance (linear) - $O(n^2)$: Speed gets much slower with distance (quadratic)

The Basics:

```
O(1)     = Always same time
O(n)     = Time grows with input
O(n²)    = Time grows much faster
```

3.4 O(1) - Constant Time

Definition: Same time regardless of input size

Examples:

```
# Example 1: Access array element
def get_first(arr):
    return arr[0]  # Always 1 operation
# O(1)

# Example 2: Simple math
def add_numbers(a, b):
    return a + b  # Always 1 operation
# O(1)

# Example 3: Multiple constant operations
def example(arr):
    first = arr[0]   # 1 operation
    last = arr[-1]   # 1 operation
    sum = first + last  # 1 operation
    return sum
# Still O(1) - constant number of operations
```

Visual:

```
Input Size:  10    100    1000    10000
Time:        1     1      1       1      ← Always same!
```

3.5 O(n) - Linear Time

Definition: Time grows proportionally with input

Examples:

```python
# Example 1: Print all elements
def print_all(arr):
    for item in arr:  # n times
        print(item)
# O(n)

# Example 2: Find maximum
def find_max(arr):
    max_val = arr[0]
    for num in arr:  # Must check all n elements
        if num > max_val:
            max_val = num
    return max_val
# O(n)

# Example 3: Sum all elements
def sum_all(arr):
    total = 0
    for num in arr:  # n operations
        total += num
    return total
# O(n)
```

Visual:

```
Input Size:  10    100    1000    10000
Time:        10    100    1000    10000   ← Grows linearly
```

Real-World Analogy: Reading a book: If one book takes 1 hour, ten books take 10 hours.

3.6 O(n²) - Quadratic Time

Definition: Time grows as square of input

Examples:

```python
# Example 1: Print all pairs
def print_pairs(arr):
    for i in arr:         # n times
```

```
        for j in arr:     # n times for each i
            print(i, j)
# O(n²) - nested loops!

# Example 2: Check for duplicates (brute force)
def has_duplicates(arr):
    for i in range(len(arr)):           # n times
        for j in range(i+1, len(arr)):  # n times
            if arr[i] == arr[j]:
                return True
    return False
# O(n²)

# Example 3: Bubble sort (we'll learn later)
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
# O(n²)
```

Visual:

```
Input Size:  10      100      1000      10000
Time:        100     10,000   1,000,000 100,000,000  ← Grows FAST!
```

Real-World Analogy: Handshakes: In a group of n people, if everyone shakes hands with everyone else, that's n^2 handshakes!

3.7 O(log n) - Logarithmic Time

Definition: Time grows slowly (halving each step)

The Best Example: Binary Search

```
def binary_search(arr, target):
    """Search in a SORTED array by halving"""
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1   # Search right half
        else:
            right = mid - 1  # Search left half
```

```
    return -1
# O(log n) - halving each time!
```

How It Works:

```
Array: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
Target: 13

Step 1: Check middle (9) - too small, go right
        [11, 13, 15, 17, 19]

Step 2: Check middle (15) - too big, go left
        [11, 13]

Step 3: Check middle (13) - found it!

Only 3 steps for 10 items!
```

Visual:

```
Input Size:  10     100     1000     10000
Time:        ~3     ~7      ~10      ~13     ← Grows very slowly!
```

Real-World Analogy: Dictionary: Open to middle, go left or right, repeat. Much faster than reading every word!

3.8 Comparing Growth Rates

From Fastest to Slowest:

```
O(1)      < O(log n)  < O(n)      < O(n log n) < O(n²)
Constant  < Logarithmic < Linear  < Linearithmic < Quadratic
```

Visual Comparison:

```
For n = 100:

O(1):      1           Super fast!
O(log n):  ~7          Very fast!
O(n):      100         Fast
O(n log n): ~700       Pretty good
O(n²):     10,000     ⚠ Getting slow
O(2^n):    1.27e30     Extremely slow!
```

Rule of Thumb: - □ O(1), O(log n), O(n): Good for large data - ⚠ O(n log n): Acceptable for large data - ⚠ O(n²): Only good for small data - □ O(2^n): Only works for tiny data

3.9 Calculating Big O (Simple Rules)

Rule 1: Drop Constants

```
def example(arr):
    print(arr[0])      # 1
    print(arr[1])      # 1
    print(arr[2])      # 1
    for item in arr:   # n
        print(item)

# Total: 3 + n operations
# Big O: O(n)  ← Drop the 3!
```

Rule 2: Drop Smaller Terms

```
def example(arr):
    for i in arr:           # n
        print(i)

    for i in arr:           # n
        for j in arr:       # n
            print(i, j)

# Total: n + n² operations
# Big O: O(n²)  ← Drop the n, keep n²!
```

Rule 3: Different Inputs = Different Variables

```
def example(arr1, arr2):
    for item in arr1:   # n operations
        print(item)

    for item in arr2:   # m operations
        print(item)

# Big O: O(n + m)  ← NOT O(n)!
```

Rule 4: Nested Loops Usually Mean Multiply

```
def example(arr):
    for i in arr:       # n times
        for j in arr:   # n times
            print(i, j)

# Big O: O(n × n) = O(n²)
```

3.10 Space Complexity (Memory Usage)

Space complexity = How much extra memory we use

O(1) - Constant Space:

```python
def sum_array(arr):
    total = 0  # Only 1 variable
    for num in arr:
        total += num
    return total
# Space: O(1) - same memory regardless of input size
```

O(n) - Linear Space:

```python
def copy_array(arr):
    new_arr = []
    for num in arr:
        new_arr.append(num)  # Creating array of size n
    return new_arr
# Space: O(n) - memory grows with input
```

Trade-offs:

```python
# Version 1: Fast but uses memory
def has_duplicates_v1(arr):
    seen = set()  # Extra memory: O(n)
    for num in arr:
        if num in seen:
            return True
        seen.add(num)
    return False
# Time: O(n), Space: O(n)

# Version 2: Slow but saves memory
def has_duplicates_v2(arr):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
# Time: O(n²), Space: O(1)
```

Sometimes we trade space for time!

3.11 Practice: Analyze These Functions

Exercise 1:

```python
def mystery1(arr):
    return arr[len(arr) // 2]
```

Answer: O(1) - Just accessing one element

Exercise 2:

```
def mystery2(arr):
    for i in range(len(arr)):
        print(arr[i])

    for i in range(len(arr)):
        print(arr[i])
```

Answer: O(n) - Two loops but not nested: n + n = 2n = O(n)

Exercise 3:

```
def mystery3(arr):
    for i in range(len(arr)):
        for j in range(100):
            print(arr[i])
```

Answer: O(n) - Inner loop is constant (100), so n × 100 = O(n)

Exercise 4:

```
def mystery4(n):
    i = 1
    while i < n:
        print(i)
        i = i * 2
```

Answer: O(log n) - Doubling each time (inverse of halving)

Chapter 3 Summary

What You Learned: - Time complexity measures how algorithms scale - Big O notation describes worst-case growth - O(1) is constant, O(n) is linear, O(n^2) is quadratic - O(log n) is very efficient (binary search) - Space complexity measures memory usage

Remember: - Faster is better - O(1) and O(log n) are excellent - O(n) is good - O(n^2) is acceptable for small inputs only - Always aim for the best complexity

Complexity Ranking:

```
FAST
  ↑
O(1)
O(log n)
O(n)
O(n log n)
O(n²)
  ↓
SLOW
```

Chapter 3 Exercises

Exercise 1: What is the time complexity?

```
def func(arr):
    print(arr[0])
    print(arr[len(arr)-1])
```

Exercise 2: What is the time complexity?

```
def func(arr):
    for i in range(len(arr)):
        if arr[i] == 5:
            return True
    return False
```

Exercise 3: What is the time complexity?

```
def func(arr):
    for i in range(len(arr)):
        for j in range(len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

Exercise 4: Which is faster for n=1000: O(n) or O(n^2)?

Exercise 5: If an algorithm takes 1 second for n=100, and it's O(n^2), approximately how long for n=1000?

Chapter 4: Basic Problem-Solving Techniques

4.1 The Problem-Solving Recipe

Every problem should follow these steps:

```
1. READ & UNDERSTAND
   ↓
2. TRY EXAMPLES
   ↓
3. THINK OF SOLUTION
   ↓
4. WRITE CODE
   ↓
5. TEST IT
```

Let's learn each step!

4.2 Step 1: Read & Understand

Questions to Ask:

1. **What is the input?**
   - Type? (number, list, string)
   - Size? (how big?)
   - Can it be empty?
2. **What is the output?**
   - What should I return?
   - Format?
3. **What are the rules?**
   - Special conditions?
   - What's not allowed?

Example Problem:

```
Find the largest number in a list.

Questions:
 Input: What type? → List of numbers
 Input: Can be empty? → No, at least 1 number
 Input: Negative numbers? → Yes
 Output: Return what? → The largest number
 Rules: Any special cases? → No

Now I understand the problem!
```

4.3 Step 2: Try Examples

Why Examples Help: - Makes problem concrete - Helps find patterns - Catches misunderstandings

Example Problem: Find Largest Number

```
Example 1: [5, 2, 8, 1, 9]
Expected: 9 ✓

Example 2: [1]
Expected: 1 ✓

Example 3: [-5, -2, -8, -1]
Expected: -1 ✓

Example 4: [3, 3, 3]
Expected: 3 ✓
```

Try at least 3-4 examples!

4.4 Step 3: Think of Solution

Start Simple:

```
Problem: Find largest number

My thinking:
1. Look at first number
2. Is it the biggest so far? Yes!
3. Look at second number
4. Is it bigger than current biggest? Maybe update
5. Continue for all numbers
6. Return the biggest

This works!
```

Write Steps in Plain English:

```
Algorithm: FindLargest
1. Set largest = first number
2. For each number in list:
   a. If number > largest:
      - Update largest = number
3. Return largest
```

4.5 Step 4: Write Code

Translate Steps to Code:

```python
# Python
def find_largest(numbers):
    # Step 1: Set largest = first number
    largest = numbers[0]

    # Step 2: For each number
    for number in numbers:
        # If number > largest
        if number > largest:
            # Update largest
            largest = number

    # Step 3: Return largest
    return largest
```

```javascript
// JavaScript
function findLargest(numbers) {
    // Step 1: Set largest = first number
    let largest = numbers[0];

    // Step 2: For each number
    for (let number of numbers) {
```

```
        // If number > largest
        if (number > largest) {
            // Update largest
            largest = number;
        }
    }

    // Step 3: Return largest
    return largest;
}
```

Tips for Clean Code: 1. Use meaningful names (`largest` not `x`) 2. Add comments 3. One step at a time 4. Keep it simple!

4.6 Step 5: Test It

Test Your Examples:

```
def find_largest(numbers):
    largest = numbers[0]
    for number in numbers:
        if number > largest:
            largest = number
    return largest

# Test cases
print(find_largest([5, 2, 8, 1, 9]))  # Should be 9
print(find_largest([1]))              # Should be 1
print(find_largest([-5, -2, -8, -1])) # Should be -1
print(find_largest([3, 3, 3]))        # Should be 3
```

All tests pass? Great!

4.7 Common Beginner Patterns

Pattern 1: Finding Something

Template:

```
def find_something(arr):
    for item in arr:
        if condition(item):
            return item  # Found it!
    return None  # Not found
```

Example: Find first even number

```
def find_first_even(numbers):
    for num in numbers:
        if num % 2 == 0:  # Condition: even
            return num
```

```
    return None
```

Pattern 2: Counting

Template:

```
def count_something(arr):
    count = 0
    for item in arr:
        if condition(item):
            count += 1
    return count
```

Example: Count negative numbers

```
def count_negatives(numbers):
    count = 0
    for num in numbers:
        if num < 0:  # Condition: negative
            count += 1
    return count
```

Pattern 3: Collecting/Filtering

Template:

```
def collect_something(arr):
    result = []
    for item in arr:
        if condition(item):
            result.append(item)
    return result
```

Example: Get all even numbers

```
def get_evens(numbers):
    evens = []
    for num in numbers:
        if num % 2 == 0:
            evens.append(num)
    return evens
```

Pattern 4: Transforming

Template:

```
def transform(arr):
    result = []
    for item in arr:
        result.append(transform_item(item))
```

```
    return result
```

Example: Square all numbers

```
def square_all(numbers):
    squared = []
    for num in numbers:
        squared.append(num * num)
    return squared
```

4.8 Complete Example Walkthrough

Problem: Remove Duplicates from List

Step 1: Understand

```
Input: List of numbers (can have duplicates)
Output: List without duplicates
Rules: Keep first occurrence only

Example: [1, 2, 2, 3, 1, 4] → [1, 2, 3, 4]
```

Step 2: Examples

```
[1, 2, 2, 3] → [1, 2, 3]
[5, 5, 5] → [5]
[1, 2, 3] → [1, 2, 3] (no duplicates)
[] → []
```

Step 3: Think

```
Idea: Keep track of what we've seen

1. Create empty list for result
2. Create empty set for seen numbers
3. For each number:
   a. If not seen before:
      - Add to result
      - Mark as seen
4. Return result
```

Step 4: Code

```
def remove_duplicates(numbers):
    result = []
    seen = set()

    for num in numbers:
        if num not in seen:
```

```
            result.append(num)
            seen.add(num)

    return result
```

Step 5: Test

```
print(remove_duplicates([1, 2, 2, 3, 1, 4]))  # [1, 2, 3, 4] ✓
print(remove_duplicates([5, 5, 5]))           # [5] ✓
print(remove_duplicates([1, 2, 3]))           # [1, 2, 3] ✓
print(remove_duplicates([]))                  # [] ✓
```

4.9 Debugging Tips

When Your Code Doesn't Work:

1. **Print Everything!**

```
def find_max(arr):
    max_val = arr[0]
    print(f"Starting with: {max_val}")  # Debug print

    for num in arr:
        print(f"Checking {num}, current max: {max_val}")  # Debug
        if num > max_val:
            max_val = num
            print(f"New max: {max_val}")  # Debug

    return max_val
```

2. **Test with Small Inputs**

```
# Instead of testing with [1, 5, 2, 9, 3, 7, 4]
# Test with [1, 5, 2] first - easier to trace!
```

3. **Check Edge Cases**

```
# Test these:
- Empty input
- Single element
- All same elements
- Negative numbers
- Very large numbers
```

4. **Read Error Messages**

```
# IndexError: list index out of range
# → You're trying to access arr[i] when i is too big!
```

```
# TypeError: unsupported operand type(s)
# → You're trying to add/compare wrong types!
```

4.10 Common Mistakes to Avoid

Mistake 1: Off-by-One Errors

```
# WRONG - misses last element!
for i in range(len(arr) - 1):
    print(arr[i])

# CORRECT
for i in range(len(arr)):
    print(arr[i])
```

Mistake 2: Not Handling Empty Input

```
# WRONG - crashes if empty!
def get_first(arr):
    return arr[0]

# CORRECT
def get_first(arr):
    if not arr:
        return None
    return arr[0]
```

Mistake 3: Modifying While Looping

```
# WRONG - can cause issues!
arr = [1, 2, 3, 4]
for num in arr:
    if num % 2 == 0:
        arr.remove(num)  # Dangerous!

# CORRECT - create new list
arr = [1, 2, 3, 4]
arr = [num for num in arr if num % 2 != 0]
```

Mistake 4: Comparing with Wrong Type

```
# WRONG
age = "25"  # String!
if age > 18:  # Comparing string with number!
    print("Adult")

# CORRECT
age = int("25")  # Convert to number
if age > 18:
```

```
    print("Adult")
```

Chapter 4 Summary

What You Learned: - 5-step problem-solving process - Always start with examples - Think before coding - Common patterns: find, count, collect, transform - Debug with print statements - Avoid common mistakes

Problem-Solving Checklist: - [ ] Understand the problem - [ ] Try 3-4 examples - [ ] Write steps in English - [ ] Code one step at a time - [ ] Test with your examples - [ ] Test edge cases

Chapter 4 Exercises

Exercise 1: Find Minimum Write a function to find the smallest number in a list.

```
# Input: [5, 2, 8, 1, 9]
# Output: 1
```

Exercise 2: Contains Value Write a function to check if a value exists in a list.

```
# Input: [1, 2, 3, 4], target=3
# Output: True
```

Exercise 3: Sum of Evens Write a function to sum all even numbers in a list.

```
# Input: [1, 2, 3, 4, 5, 6]
# Output: 12 (2+4+6)
```

Exercise 4: Reverse List Write a function to reverse a list without using built-in reverse.

```
# Input: [1, 2, 3, 4]
# Output: [4, 3, 2, 1]
```

Exercise 5: Count Vowels Write a function to count vowels in a string.

```
# Input: "hello"
# Output: 2 ('e', 'o')
```

Module 2: Basic Data Structures

Chapter 5: Arrays - Your First Data Structure

5.1 What is an Array?

An **array** is a collection of items stored together in memory.

Real-World Analogy:

```
Array = A row of lockers

Locker 0: [Book]
Locker 1: [Pencil]
Locker 2: [Notebook]
Locker 3: [Eraser]
Locker 4: [Ruler]

Each locker has:
- A number (index): 0, 1, 2, 3, 4
- Something inside (value)
```

Visual Representation:

```
Index:    0    1    2    3    4
Array:  [10] [20] [30] [40] [50]
```

Why Arrays Are Useful: - ✅ Fast access to any element (O(1)) - ✅ Simple to understand - ✅ Store multiple values in one variable - ❌ Fixed size (in some languages) - ❌ Slow to insert/delete in middle

5.2 Creating Arrays

Python (Lists):

```python
# Empty array
arr = []

# Array with values
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "orange"]
mixed = [1, "hello", 3.14, True]  # Python allows mixing types!

# Array of specific size (all zeros)
zeros = [0] * 10  # [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

JavaScript:

```javascript
// Empty array
let arr = [];

// Array with values
let numbers = [1, 2, 3, 4, 5];
let fruits = ["apple", "banana", "orange"];
let mixed = [1, "hello", 3.14, true];  // JS allows mixing too!
```

```
// Array of specific size
let zeros = new Array(10).fill(0);
```

5.3 Accessing Array Elements

Indexing starts at 0!

```
# Python
fruits = ["apple", "banana", "orange", "grape"]

first = fruits[0]   # "apple"
second = fruits[1]  # "banana"
third = fruits[2]   # "orange"

# Negative indexing (from end)
last = fruits[-1]   # "grape"
second_last = fruits[-2]  # "orange"

# Time: O(1) - instant access!
```

```
// JavaScript
let fruits = ["apple", "banana", "orange", "grape"];

let first = fruits[0];   // "apple"
let second = fruits[1];  // "banana"
let third = fruits[2];   // "orange"

// Access from end
let last = fruits[fruits.length - 1];  // "grape"

// Time: O(1)
```

5.4 Array Length

```
# Python
numbers = [1, 2, 3, 4, 5]
size = len(numbers)  # 5
```

```
// JavaScript
let numbers = [1, 2, 3, 4, 5];
let size = numbers.length;  // 5
```

5.5 Adding Elements

Add to End (Fast!)

```python
# Python
arr = [1, 2, 3]
arr.append(4)  # [1, 2, 3, 4]
arr.append(5)  # [1, 2, 3, 4, 5]
# Time: O(1) amortized
```

```javascript
// JavaScript
let arr = [1, 2, 3];
arr.push(4);  // [1, 2, 3, 4]
arr.push(5);  // [1, 2, 3, 4, 5]
// Time: O(1) amortized
```

Add to Beginning (Slow!)

```python
# Python
arr = [2, 3, 4]
arr.insert(0, 1)  # [1, 2, 3, 4]
# Time: O(n) - must shift all elements!

# Visual:
# Before: [2, 3, 4]
# Shift:  [ ][2][3][4]
# Insert: [1][2][3][4]
```

```javascript
// JavaScript
let arr = [2, 3, 4];
arr.unshift(1);  // [1, 2, 3, 4]
// Time: O(n)
```

Add to Middle

```python
# Python
arr = [1, 2, 4, 5]
arr.insert(2, 3)  # [1, 2, 3, 4, 5]
# Insert 3 at index 2
# Time: O(n)
```

```javascript
// JavaScript
let arr = [1, 2, 4, 5];
arr.splice(2, 0, 3);  // [1, 2, 3, 4, 5]
// Insert 3 at index 2
// Time: O(n)
```

5.6 Removing Elements

Remove from End (Fast!)

```python
# Python
arr = [1, 2, 3, 4, 5]
last = arr.pop()  # Returns 5, arr is now [1, 2, 3, 4]
# Time: O(1)
```

```javascript
// JavaScript
let arr = [1, 2, 3, 4, 5];
let last = arr.pop();  // Returns 5, arr is [1, 2, 3, 4]
// Time: O(1)
```

Remove from Beginning (Slow!)

```python
# Python
arr = [1, 2, 3, 4]
arr.pop(0)  # Removes 1, arr is [2, 3, 4]
# Time: O(n)
```

```javascript
// JavaScript
let arr = [1, 2, 3, 4];
arr.shift();  // Removes 1, arr is [2, 3, 4]
// Time: O(n)
```

Remove Specific Value

```python
# Python
arr = [1, 2, 3, 2, 4]
arr.remove(2)  # Removes first 2, arr is [1, 3, 2, 4]
# Time: O(n) - must find it first
```

Remove by Index

```python
# Python
arr = [1, 2, 3, 4, 5]
del arr[2]  # Removes index 2, arr is [1, 2, 4, 5]
# Time: O(n)
```

```javascript
// JavaScript
let arr = [1, 2, 3, 4, 5];
arr.splice(2, 1);  // Removes index 2, arr is [1, 2, 4, 5]
// Time: O(n)
```

5.7 Looping Through Arrays

Method 1: For Loop with Index

```python
# Python
fruits = ["apple", "banana", "orange"]
```

```python
for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")

# Output:
# Index 0: apple
# Index 1: banana
# Index 2: orange
```

```javascript
// JavaScript
let fruits = ["apple", "banana", "orange"];

for (let i = 0; i < fruits.length; i++) {
    console.log(`Index ${i}: ${fruits[i]}`);
}
```

Method 2: For-Each Loop

```python
# Python
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# orange
```

```javascript
// JavaScript
let fruits = ["apple", "banana", "orange"];

for (let fruit of fruits) {
    console.log(fruit);
}
```

Method 3: While Loop

```python
# Python
fruits = ["apple", "banana", "orange"]
i = 0

while i < len(fruits):
    print(fruits[i])
    i += 1
```

```javascript
// JavaScript
let fruits = ["apple", "banana", "orange"];
```

```
let i = 0;

while (i < fruits.length) {
    console.log(fruits[i]);
    i++;
}
```

5.8 Common Array Operations

Finding an Element

```
# Python
numbers = [10, 20, 30, 40, 50]

# Check if exists
if 30 in numbers:
    print("Found!")  # Found!

# Get index
index = numbers.index(30)  # 2

# Time: O(n) - must search entire array
```

```
// JavaScript
let numbers = [10, 20, 30, 40, 50];

// Check if exists
if (numbers.includes(30)) {
    console.log("Found!");
}

// Get index
let index = numbers.indexOf(30);  // 2

// Time: O(n)
```

Slicing (Getting Part of Array)

```
# Python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Get elements from index 2 to 5 (exclusive)
subset = numbers[2:5]  # [2, 3, 4]

# Get first 3
first_three = numbers[:3]  # [0, 1, 2]

# Get last 3
last_three = numbers[-3:]  # [7, 8, 9]
```

```python
# Every other element
evens = numbers[::2]  # [0, 2, 4, 6, 8]
```

```javascript
// JavaScript
let numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];

// Get elements from index 2 to 5 (exclusive)
let subset = numbers.slice(2, 5);  // [2, 3, 4]

// Get first 3
let firstThree = numbers.slice(0, 3);  // [0, 1, 2]

// Get last 3
let lastThree = numbers.slice(-3);  // [7, 8, 9]
```

Sorting

```python
# Python
numbers = [3, 1, 4, 1, 5, 9, 2]

# Sort in place
numbers.sort()  # [1, 1, 2, 3, 4, 5, 9]

# Create sorted copy
original = [3, 1, 4]
sorted_copy = sorted(original)  # [1, 3, 4]
# original is still [3, 1, 4]
```

```javascript
// JavaScript
let numbers = [3, 1, 4, 1, 5, 9, 2];

// Sort in place
numbers.sort((a, b) => a - b);  // [1, 1, 2, 3, 4, 5, 9]
```

Reversing

```python
# Python
arr = [1, 2, 3, 4, 5]

# Reverse in place
arr.reverse()  # [5, 4, 3, 2, 1]

# Create reversed copy
original = [1, 2, 3]
reversed_copy = original[::-1]  # [3, 2, 1]
```

```javascript
// JavaScript
let arr = [1, 2, 3, 4, 5];
```

```
// Reverse in place
arr.reverse();  // [5, 4, 3, 2, 1]
```

5.9 Your First Array Problems

Problem 1: Sum All Elements

```
# Python
def sum_array(arr):
    """Add all numbers in array"""
    total = 0
    for num in arr:
        total += num
    return total

# Test
print(sum_array([1, 2, 3, 4, 5]))  # 15
# Time: O(n), Space: O(1)
```

Problem 2: Find Maximum Element

```
# Python
def find_max(arr):
    """Find largest number"""
    if not arr:
        return None

    max_val = arr[0]  # Assume first is largest

    for num in arr:
        if num > max_val:
            max_val = num  # Found new max!

    return max_val

# Test
print(find_max([3, 7, 2, 9, 1]))  # 9
# Time: O(n), Space: O(1)
```

Problem 3: Count Occurrences

```
# Python
def count_occurrences(arr, target):
    """Count how many times target appears"""
    count = 0
    for num in arr:
        if num == target:
            count += 1
    return count
```

```python
# Test
print(count_occurrences([1, 2, 3, 2, 4, 2], 2))  # 3
# Time: O(n), Space: O(1)
```

Problem 4: Reverse Array

```python
# Python
def reverse_array(arr):
    """Reverse array using two pointers"""
    left = 0
    right = len(arr) - 1

    while left < right:
        # Swap elements
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

    return arr

# Test
print(reverse_array([1, 2, 3, 4, 5]))  # [5, 4, 3, 2, 1]
# Time: O(n), Space: O(1)
```

Problem 5: Remove Duplicates (Sorted Array)

```python
# Python
def remove_duplicates_sorted(arr):
    """Remove duplicates from sorted array
    [1, 1, 2, 2, 3] → [1, 2, 3]
    """
    if not arr:
        return []

    # Write pointer starts at 1
    write = 1

    # Read through array
    for read in range(1, len(arr)):
        # If different from previous
        if arr[read] != arr[read-1]:
            arr[write] = arr[read]
            write += 1

    # Return only unique part
    return arr[:write]

# Test
print(remove_duplicates_sorted([1, 1, 2, 2, 3, 3, 3]))  # [1, 2, 3]
# Time: O(n), Space: O(1)
```

Chapter 5 Summary

What You Learned: - Arrays store multiple values together - Access is O(1) - very fast! - Adding/removing at end: O(1) - Adding/removing at beginning/middle: O(n) - Searching: O(n) without sorting - Common operations: loop, find, sort, reverse

Array Operations Cheat Sheet:

```
Access arr[i]:      O(1)
Append to end:      O(1)
Insert at start:    O(n) ⚠
Remove from end:    O(1)
Remove from start:  O(n) ⚠
Search:             O(n) ⚠
```

Chapter 5 Exercises

Exercise 1: Find the second largest number in an array

Exercise 2: Check if array is sorted in ascending order

Exercise 3: Rotate array right by k positions

```
# Example: [1,2,3,4,5], k=2 → [4,5,1,2,3]
```

Exercise 4: Find the missing number in array [0…n]

```
# Example: [0,1,3,4] → missing 2
```

Exercise 5: Merge two sorted arrays into one sorted array

[Continues with Chapters 6-12 covering Strings, Linked Lists, Stacks, Queues, Simple Sorting, Simple Searching, and Introduction to Recursion…]

BEGINNER GUIDE COMPLETION ROADMAP

After finishing all chapters, you'll be ready for: - Basic coding interviews - Intermediate DSA course - Solving 100+ easy LeetCode problems - Understanding common algorithms

Next Steps: 1. Practice 30-50 easy problems on LeetCode 2. Review all chapter exercises 3. Move to Intermediate Guide 4. Keep coding daily!

Appendix A: Beginner's Big O Cheat Sheet

Time Complexities

```
O(1)     - Constant     - Array access: arr[0]
O(log n) - Logarithmic  - Binary search (later)
O(n)     - Linear       - Loop through array
O(n²)    - Quadratic    - Nested loops
```

When to Use What

```
Need fast access?        → Array
Need fast insert/delete? → Linked List (next)
Need LIFO (last in, first out)? → Stack (coming)
Need FIFO (first in, first out)? → Queue (coming)
```

Appendix B: 50 Easy Practice Problems

Arrays (15 problems)

1. Find largest element
2. Find smallest element
3. Sum of array
4. Average of array
5. Count even numbers
6. Count odd numbers
7. Reverse array
8. Check if sorted
9. Remove duplicates
10. Find second largest
11. Rotate array
12. Missing number
13. Two sum (given later)
14. Contains duplicate
15. Majority element

Strings (15 problems)

16-30. [Coming in Chapter 6]

Basic Logic (10 problems)

31-40. [Coming in Chapter 7-9]

Sorting/Searching (10 problems)

41-50. [Coming in Chapter 10-11]

Appendix C: Common Mistakes Beginners Make

1. **Off-by-one errors**
   - Using `range(len(arr)-1)` instead of `range(len(arr))`
2. **Not handling empty input**
   - Always check `if not arr:` first!
3. **Modifying array while looping**
   - Don't do: `for x in arr: arr.remove(x)`
4. **Confusing index with value**

   ```
   arr = [10, 20, 30]
   # arr[1] is 20, not 10!
   # Index starts at 0!
   ```

5. **Not testing edge cases**
   - Test: empty, single element, all same, negative numbers

Appendix D: Exercise Solutions

Chapter 1 Solutions:

```
# Exercise 1: Average
def average(arr):
    return sum(arr) / len(arr)

# Exercise 2: Find Minimum
def find_min(arr):
    min_val = arr[0]
    for num in arr:
        if num < min_val:
            min_val = num
    return min_val

# Exercise 3: Count Odd
def count_odd(arr):
    count = 0
    for num in arr:
        if num % 2 != 0:
```

```
            count += 1
    return count

# Exercise 4: Product
def product(arr):
    result = 1
    for num in arr:
        result *= num
    return result

# Exercise 5: Number Exists
def exists(arr, target):
    for num in arr:
        if num == target:
            return True
    return False
```

Glossary for Beginners

Algorithm: Step-by-step instructions to solve a problem

Array: Collection of items stored together

Big O: Way to describe how fast/slow an algorithm is

Index: Position of element in array (starts at 0!)

Iteration: Going through elements one by one (looping)

Time Complexity: How speed changes as data grows

Space Complexity: How memory usage changes as data grows

Resources for Continued Learning

Practice Websites

- LeetCode.com (Easy problems)
- HackerRank.com
- Codewars.com

Video Tutorials

- FreeCodeCamp DSA Course
- CS50 Introduction to Computer Science

Books

- “Grokking Algorithms” by Aditya Bhargava (very beginner-friendly!)

Congratulations on starting your DSA journey! Keep practicing, stay curious, and don’t give up. Every expert was once a beginner. You’ve got this!

END OF BEGINNER GUIDE - Part 1 *Chapters 6-12 continue with Strings, Linked Lists, Stacks, Queues, and Simple Algorithms*